

OLA Ensemble Learning

November 18, 2025

0.1 Ola - Ensemble Learning

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
from scipy import stats
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn import metrics
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import precision_recall_curve
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import MinMaxScaler
from datetime import datetime
from statsmodels.stats.outliers_influence import variance_inflation_factor
import warnings
warnings.filterwarnings("ignore")
```

```
[3]: ola = pd.read_csv("ola_driver_scaler.csv")
```

```
[4]: ola.head()
```

```
[4]: Unnamed: 0    MMM-YY  Driver_ID  Age  Gender  City  Education_Level  \
0          0    01/01/19          1  28.0    0.0  C23                2
1          1    02/01/19          1  28.0    0.0  C23                2
2          2    03/01/19          1  28.0    0.0  C23                2
3          3   11/01/20          2  31.0    0.0   C7                2
4          4   12/01/20          2  31.0    0.0   C7                2

      Income  Dateofjoining  LastWorkingDate  Joining  Designation  Grade  \
0    57387      24/12/18              NaN          1          1
1    57387      24/12/18              NaN          1          1
2    57387      24/12/18      03/11/19          1          1
3    67016      11/06/20              NaN          2          2
4    67016      11/06/20              NaN          2          2
```

	Total Business Value	Quarterly Rating
0	2381060	2
1	-665480	2
2	0	2
3	0	1
4	0	1

EXPLORATORY DATA ANALYSIS

```
[5]: print('Rows in the ola dataset: ',ola.shape[0])
      print('Columns in the ola dataset: ',ola.shape[1])
```

Rows in the ola dataset: 19104

Columns in the ola dataset: 14

```
[6]: ola.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            19104 non-null  int64
1   MMM-YY                19104 non-null  object
2   Driver_ID             19104 non-null  int64
3   Age                   19043 non-null  float64
4   Gender                19052 non-null  float64
5   City                  19104 non-null  object
6   Education_Level       19104 non-null  int64
7   Income                19104 non-null  int64
8   Dateofjoining         19104 non-null  object
9   LastWorkingDate       1616 non-null   object
10  Joining Designation    19104 non-null  int64
11  Grade                 19104 non-null  int64
12  Total Business Value  19104 non-null  int64
13  Quarterly Rating      19104 non-null  int64
dtypes: float64(2), int64(8), object(4)
memory usage: 2.0+ MB
```

Column Profiling: • MMMM-YY : Reporting Date (Monthly) (date-time)

- Age : Age of the employee (numerical)
- Gender : Gender of the employee – Male : 0, Female: 1 (categorical)
- City : City Code of the employee (categorical)
- Education_Level : Education level – 0 for 10+ ,1 for 12+ ,2 for graduate (categorical)
- Income : Monthly average Income of the employee (numerical)
- Date Of Joining : Joining date for the employee (date-time)

- LastWorkingDate : Last date of working for the employee - Target Feature (date-time, but will be converted to categorical)
- Joining Designation : Designation of the employee at the time of joining (categorical, ordinal)
- Grade : Grade of the employee at the time of reporting (categorical, ordinal)
- Total Business Value : The total business value acquired by the employee in a month (negative business indicates cancellation/refund or car EMI adjustments) (numerical)
- Quarterly Rating : Quarterly rating of the employee: 1,2,3,4,5 (categorical, ordinal - higher is better)

```
[7]: ola.describe()
```

```
[7]:
```

	Unnamed: 0	Driver_ID	Age	Gender	\
count	19104.000000	19104.000000	19043.000000	19052.000000	
mean	9551.500000	1415.591133	34.668435	0.418749	
std	5514.994107	810.705321	6.257912	0.493367	
min	0.000000	1.000000	21.000000	0.000000	
25%	4775.750000	710.000000	30.000000	0.000000	
50%	9551.500000	1417.000000	34.000000	0.000000	
75%	14327.250000	2137.000000	39.000000	1.000000	
max	19103.000000	2788.000000	58.000000	1.000000	

	Education_Level	Income	Joining Designation	Grade	\
count	19104.000000	19104.000000	19104.000000	19104.000000	
mean	1.021671	65652.025126	1.690536	2.252670	
std	0.800167	30914.515344	0.836984	1.026512	
min	0.000000	10747.000000	1.000000	1.000000	
25%	0.000000	42383.000000	1.000000	1.000000	
50%	1.000000	60087.000000	1.000000	2.000000	
75%	2.000000	83969.000000	2.000000	3.000000	
max	2.000000	188418.000000	5.000000	5.000000	

	Total Business Value	Quarterly Rating
count	1.910400e+04	19104.000000
mean	5.716621e+05	2.008899
std	1.128312e+06	1.009832
min	-6.000000e+06	1.000000
25%	0.000000e+00	1.000000
50%	2.500000e+05	2.000000
75%	6.997000e+05	3.000000
max	3.374772e+07	4.000000

```
[8]: ola.describe(include='object')
```

```
[8]:
```

	MMM-YY	City	Dateofjoining	LastWorkingDate
count	19104	19104	19104	1616
unique	24	29	869	493

top	01/01/19	C20	23/07/15	29/07/20
freq	1022	1008	192	70

```
[9]: # since unnamed and driver_id columns have the highest correlation and they r_u
      ↳ the same
      # here, dropping unnamed column

      ola.drop(columns='Unnamed: 0',axis=1,inplace=True)
```

```
[10]: ola.nunique()
```

```
[10]: MMM-YY                24
      Driver_ID            2381
      Age                  36
      Gender                2
      City                 29
      Education_Level       3
      Income               2383
      Dateofjoining         869
      LastWorkingDate       493
      Joining Designation    5
      Grade                 5
      Total Business Value  10181
      Quarterly Rating       4
      dtype: int64
```

```
[11]: ola.isna().sum()
```

```
[11]: MMM-YY                0
      Driver_ID            0
      Age                  61
      Gender               52
      City                 0
      Education_Level       0
      Income               0
      Dateofjoining         0
      LastWorkingDate       17488
      Joining Designation    0
      Grade                 0
      Total Business Value   0
      Quarterly Rating       0
      dtype: int64
```

```
[12]: ola.head(3)
```

```
[12]:   MMM-YY  Driver_ID  Age  Gender  City  Education_Level  Income  \
0  01/01/19         1  28.0    0.0  C23                2   57387
```

1	02/01/19	1	28.0	0.0	C23	2	57387
2	03/01/19	1	28.0	0.0	C23	2	57387

	Dateofjoining	LastWorkingDate	Joining	Designation	Grade	\
0	24/12/18	NaN	1		1	
1	24/12/18	NaN	1		1	
2	24/12/18	03/11/19	1		1	

	Total Business Value	Quarterly Rating
0	2381060	2
1	-665480	2
2	0	2

DATA PROCESSING AND FEATURE ENGINEERING

```
[13]: ola1 = ola.copy(deep=True)
```

```
[14]: ## Target variable creation: Create a column called target which tells whether
      ↳ the driver has left the company-
      ## driver whose last working day is present will have the value 1

first = (ola1.groupby('Driver_ID').agg({'LastWorkingDate':
      ↳ 'last'}))['LastWorkingDate'].isna()).reset_index()
first['LastWorkingDate'].replace({True:1,False:0},inplace=True)
first.rename(columns={'LastWorkingDate':'target'},inplace=True)
first.head()
```

```
[14]:   Driver_ID  target
0         1        0
1         2        1
2         4        0
3         5        0
4         6        1
```

```
[15]: ## Create a column which tells whether the quarterly rating has increased for
      ↳ that driver -
      ## for those whose quarterly rating has increased we assign the value 1
QR1 = (ola1.groupby('Driver_ID').agg({'Quarterly Rating':'first'}))['Quarterly_
      ↳ Rating']).reset_index()
QR2 = (ola1.groupby('Driver_ID').agg({'Quarterly Rating':'last'}))['Quarterly_
      ↳ Rating']).reset_index()
```

```
[16]: QR1.shape, QR2.shape
```

```
[16]: ((2381, 2), (2381, 2))
```

```
[17]: QR1.isna().sum(), QR2.isna().sum()
```

```
[17]: (Driver_ID      0
      Quarterly Rating  0
      dtype: int64,
      Driver_ID      0
      Quarterly Rating  0
      dtype: int64)
```

```
[18]: first = first.merge(QR1,on='Driver_ID')
      first = first.merge(QR2,on='Driver_ID')
```

```
[19]: first.head()
```

```
[19]:   Driver_ID  target  Quarterly Rating_x  Quarterly Rating_y
0         1      0         2         2
1         2      1         1         1
2         4      0         1         1
3         5      0         1         1
4         6      1         1         2
```

```
[20]: first['Promotion'] = np.where(first['Quarterly Rating_x'] == first['Quarterly_Rating_y'], 0,1)
```

```
[21]: # Create a column which tells whether the monthly income has increased for that driver -
      # for those whose monthly income has increased we assign the value 1
      incm1 = (ola1.groupby('Driver_ID').agg({'Income': 'first'})['Income']).reset_index()
      incm2 = (ola1.groupby('Driver_ID').agg({'Income': 'last'})['Income']).reset_index()
```

```
[22]: incm1.shape, incm2.shape
```

```
[22]: ((2381, 2), (2381, 2))
```

```
[23]: incm1.isna().sum(), incm2.isna().sum()
```

```
[23]: (Driver_ID      0
      Income      0
      dtype: int64,
      Driver_ID      0
      Income      0
      dtype: int64)
```

```
[24]: first = first.merge(incm1,on='Driver_ID')
      first = first.merge(incm2,on='Driver_ID')
```

```
[25]: first.head()
```

```
[25]:
```

	Driver_ID	target	Quarterly Rating_x	Quarterly Rating_y	Promotion	\
0	1	0	2	2	0	
1	2	1	1	1	0	
2	4	0	1	1	0	
3	5	0	1	1	0	
4	6	1	1	2	1	

	Income_x	Income_y
0	57387	57387
1	67016	67016
2	65603	65603
3	46368	46368
4	78728	78728

```
[26]: first['Raise'] = np.where(first['Income_x'] == first['Income_y'], 0,1)
```

```
[27]: first.head()
```

```
[27]:
```

	Driver_ID	target	Quarterly Rating_x	Quarterly Rating_y	Promotion	\
0	1	0	2	2	0	
1	2	1	1	1	0	
2	4	0	1	1	0	
3	5	0	1	1	0	
4	6	1	1	2	1	

	Income_x	Income_y	Raise
0	57387	57387	0
1	67016	67016	0
2	65603	65603	0
3	46368	46368	0
4	78728	78728	0

```
[28]: first.tail()
```

```
[28]:
```

	Driver_ID	target	Quarterly Rating_x	Quarterly Rating_y	Promotion	\
2376	2784	1	3	4	1	
2377	2785	0	1	1	0	
2378	2786	0	2	1	1	
2379	2787	0	2	1	1	
2380	2788	1	1	2	1	

	Income_x	Income_y	Raise
2376	82815	82815	0
2377	12105	12105	0
2378	35370	35370	0
2379	69498	69498	0
2380	70254	70254	0

```
[29]: first = first[['Driver_ID','target','Raise','Promotion']]
```

```
[30]: first.head()
```

```
[30]:
```

	Driver_ID	target	Raise	Promotion
0	1	0	0	0
1	2	1	0	0
2	4	0	0	0
3	5	0	0	0
4	6	1	0	1

```
[31]: functions = {'MMM-YY':'count',
                    'Driver_ID':'first',
                    'Age':'max',
                    'Gender':'last',
                    'City':'last',
                    'Education_Level':'last',
                    'Dateofjoining':'first',
                    'LastWorkingDate':'last',
                    'Grade':'last',
                    'Total Business Value':'sum',
                    'Income':'sum',
                    'Dateofjoining':'first',
                    'LastWorkingDate':'last',
                    'Joining Designation':'last',
                    'Grade':'last',
                    'Quarterly Rating':'first'}
ola1 = ola1.groupby([ola1['Driver_ID']]).aggregate(functions)
ola1['month'] = pd.to_datetime(ola1['Dateofjoining']).dt.month
ola1['year'] = pd.DatetimeIndex(ola1['Dateofjoining']).year
ola1.rename(columns={'MMM-YY':'Reportings'},inplace=True)
```

```
[32]: ola1.reset_index(drop=True, inplace=True)
ola1 = ola1.merge(first,on='Driver_ID')
ola1.head()
```

```
[32]:
```

	Reportings	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	\
0	3	1	28.0	0.0	C23	2	24/12/18	
1	2	2	31.0	0.0	C7	2	11/06/20	
2	5	4	43.0	0.0	C13	2	12/07/19	
3	3	5	29.0	0.0	C9	0	01/09/19	
4	5	6	31.0	1.0	C11	1	31/07/20	

	LastWorkingDate	Grade	Total Business Value	Income	Joining Designation	\
0	03/11/19	1	1715580	172161		1
1	None	2	0	134032		2
2	27/04/20	2	350000	328015		2

3	03/07/19	1	120360	139104	1
4	None	3	1265000	393640	3

	Quarterly Rating	month	year	target	Raise	Promotion
0	2	12	2018	0	0	0
1	1	12	2020	1	0	0
2	1	11	2019	0	0	0
3	1	12	2019	0	0	0
4	1	12	2020	1	0	1

```
[ ]: pip install regex
```

```
[43]: ola1['Age'] = ola1['Age'].astype('int64')
      ola1['Cities'] =ola1['City'].astype('str').str.extractall('(\d+)').unstack().
      ↪fillna('').sum(axis=1).astype(int)
```

```
[44]: ola1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2381 entries, 0 to 2380
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Reportings                            2381 non-null   int64
1   Driver_ID                             2381 non-null   int64
2   Age                                    2381 non-null   int64
3   Gender                                2381 non-null   float64
4   City                                   2381 non-null   object
5   Education_Level                       2381 non-null   int64
6   Dateofjoining                         2381 non-null   object
7   LastWorkingDate                       1616 non-null   object
8   Grade                                  2381 non-null   int64
9   Total Business Value                  2381 non-null   int64
10  Income                                2381 non-null   int64
11  Joining Designation                   2381 non-null   int64
12  Quarterly Rating                      2381 non-null   int64
13  month                                 2381 non-null   int32
14  year                                  2381 non-null   int32
15  target                                2381 non-null   int64
16  Raise                                 2381 non-null   int64
17  Promotion                             2381 non-null   int64
18  Cities                                2381 non-null   int64
dtypes: float64(1), int32(2), int64(13), object(3)
memory usage: 335.0+ KB
```

```
[45]: ola1.
      ↪drop(columns=['Dateofjoining','LastWorkingDate','City'],axis=1,inplace=True)
      ola1['Gender'].replace({'M':0,'F':1},inplace=True)
```

```
ola1['Gender'] = ola1['Gender'].astype('int64')
```

```
[46]: ola1.head()
```

```
[46]:
```

	Reportings	Driver_ID	Age	Gender	Education_Level	Grade	\
0	3	1	28	0	2	1	
1	2	2	31	0	2	2	
2	5	4	43	0	2	2	
3	3	5	29	0	0	1	
4	5	6	31	1	1	3	

	Total Business Value	Income	Joining Designation	Quarterly Rating	month	\
0	1715580	172161	1	2	12	
1	0	134032	2	1	12	
2	350000	328015	2	1	11	
3	120360	139104	1	1	12	
4	1265000	393640	3	1	12	

	year	target	Raise	Promotion	Cities
0	2018	0	0	0	23
1	2020	1	0	0	7
2	2019	0	0	0	13
3	2019	0	0	0	9
4	2020	1	0	1	11

```
[47]: sum(ola1.isna().sum())
```

```
[47]: 0
```

```
[48]: ola1.describe().T
```

```
[48]:
```

	count	mean	std	min	25%	\
Reportings	2381.0	8.023520e+00	6.783590e+00	1.0	3.0	
Driver_ID	2381.0	1.397559e+03	8.061616e+02	1.0	695.0	
Age	2381.0	3.366317e+01	5.983375e+00	21.0	29.0	
Gender	2381.0	4.103318e-01	4.919972e-01	0.0	0.0	
Education_Level	2381.0	1.007560e+00	8.162900e-01	0.0	0.0	
Grade	2381.0	2.096598e+00	9.415218e-01	1.0	1.0	
Total Business Value	2381.0	4.586742e+06	9.127115e+06	-1385530.0	0.0	
Income	2381.0	5.267603e+05	6.231633e+05	10883.0	139895.0	
Joining Designation	2381.0	1.820244e+00	8.414334e-01	1.0	1.0	
Quarterly Rating	2381.0	1.486350e+00	8.343483e-01	1.0	1.0	
month	2381.0	6.975220e+00	3.007801e+00	1.0	5.0	
year	2381.0	2.018536e+03	1.609597e+00	2013.0	2018.0	
target	2381.0	3.212936e-01	4.670713e-01	0.0	0.0	
Raise	2381.0	1.805964e-02	1.331951e-01	0.0	0.0	
Promotion	2381.0	3.427131e-01	4.747162e-01	0.0	0.0	

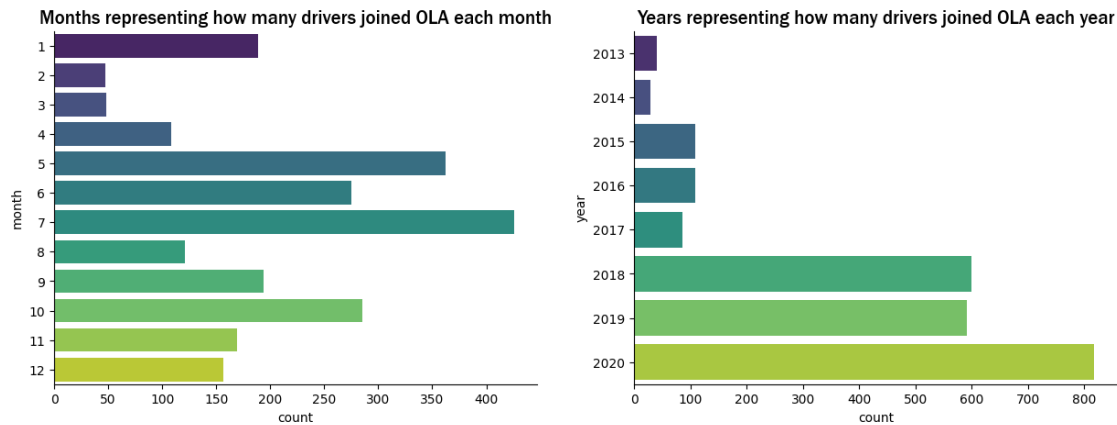
Cities	2381.0	1.533557e+01	8.371843e+00	1.0	8.0
	50%	75%	max		
Reportings	5.0	10.0	24.0		
Driver_ID	1400.0	2100.0	2788.0		
Age	33.0	37.0	58.0		
Gender	0.0	1.0	1.0		
Education_Level	1.0	2.0	2.0		
Grade	2.0	3.0	5.0		
Total Business Value	817680.0	4173650.0	95331060.0		
Income	292980.0	651456.0	4522032.0		
Joining Designation	2.0	2.0	5.0		
Quarterly Rating	1.0	2.0	4.0		
month	7.0	10.0	12.0		
year	2019.0	2020.0	2020.0		
target	0.0	1.0	1.0		
Raise	0.0	0.0	1.0		
Promotion	0.0	1.0	1.0		
Cities	15.0	22.0	29.0		

DATA VISUALIZATION

0.2 Univariate

```
[57]: fig = plt.figure(figsize=(15,5))
ax = fig.add_subplot(1,2,1)
sns.countplot(y=ola1.month,palette='viridis')
plt.title('Months representing how many drivers joined OLA each_
↪month',fontname='Franklin Gothic Medium', fontsize=15)

ax = fig.add_subplot(1,2,2)
sns.countplot(y=ola1.year,palette='viridis')
plt.title('Years representing how many drivers joined OLA each_
↪year',fontname='Franklin Gothic Medium', fontsize=15)
sns.despine()
plt.show()
```

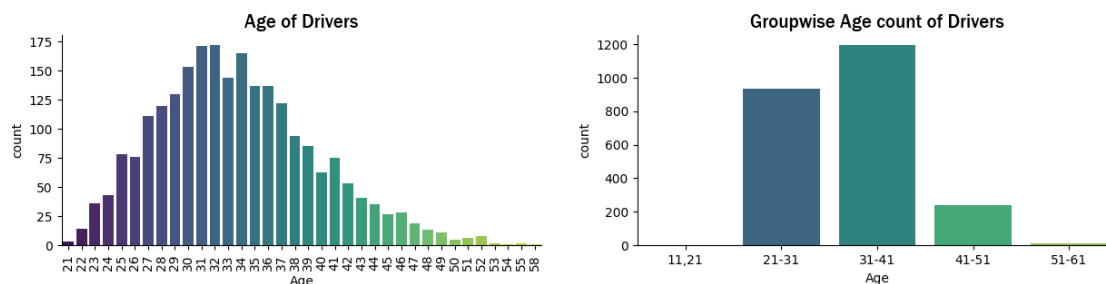


Observations:

- July received the maximum number of drivers in 8 years.
- February and March receives the least number of Drivers joining OLA.
- Joining of Drivers receives a boost of about 500% after 2017.

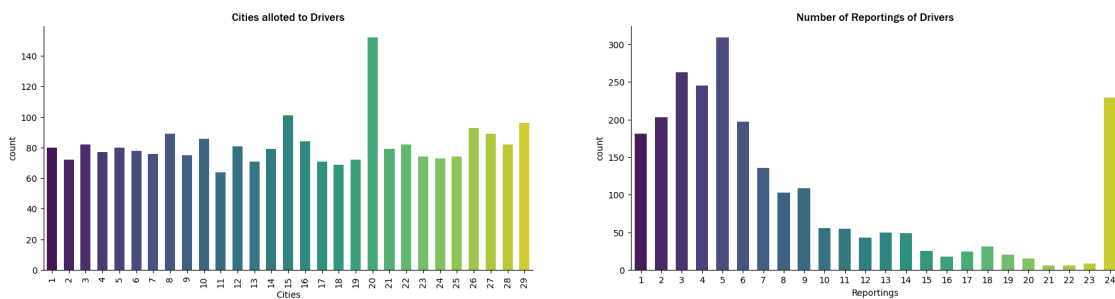
```
[58]: fig = plt.figure(figsize=(15,3))
ax = fig.add_subplot(121)
sns.countplot(x=ola1.Age,palette='viridis',width=0.8)
plt.title('Age of Drivers',fontname='Franklin Gothic Medium', fontsize=15)
plt.xticks(rotation=90)

ax = fig.add_subplot(122)
a = pd.cut(ola1.
    ↪Age,bins=[11,21,31,41,51,61],labels=['11,21','21-31','31-41','41-51','51-61'])
sns.countplot(x=a,palette='viridis')
plt.title('Groupwise Age count of Drivers',fontname='Franklin Gothic Medium',
    ↪fontsize=15)
sns.despine()
plt.show()
```



```
[59]: fig = plt.figure(figsize=(22,5))
ax = fig.add_subplot(121)
sns.countplot(x=ola1.Cities,palette='viridis',width=0.6)
plt.title('Cities allotted to Drivers',fontname='Franklin Gothic Medium',
fontsize=13)
plt.xticks(rotation=90)

ax = fig.add_subplot(122)
sns.countplot(x=ola1.Reportings,palette='viridis',width=0.6)
plt.title('Number of Reportings of Drivers',fontname='Franklin Gothic Medium',
fontsize=13)
sns.despine()
plt.show()
```



```
[60]: plt.figure(figsize=(20,13))
plt.subplot(4,2,1)
sns.countplot(x=ola1.Grade,palette='viridis')

plt.subplot(4,2,2)
sns.countplot(x=ola1['Joining Designation'],palette='viridis')

plt.subplot(4,2,3)
sns.countplot(x=ola1.Education_Level,palette='viridis')

plt.subplot(4,2,4)
sns.countplot(x=ola1['Quarterly Rating'],palette='viridis')

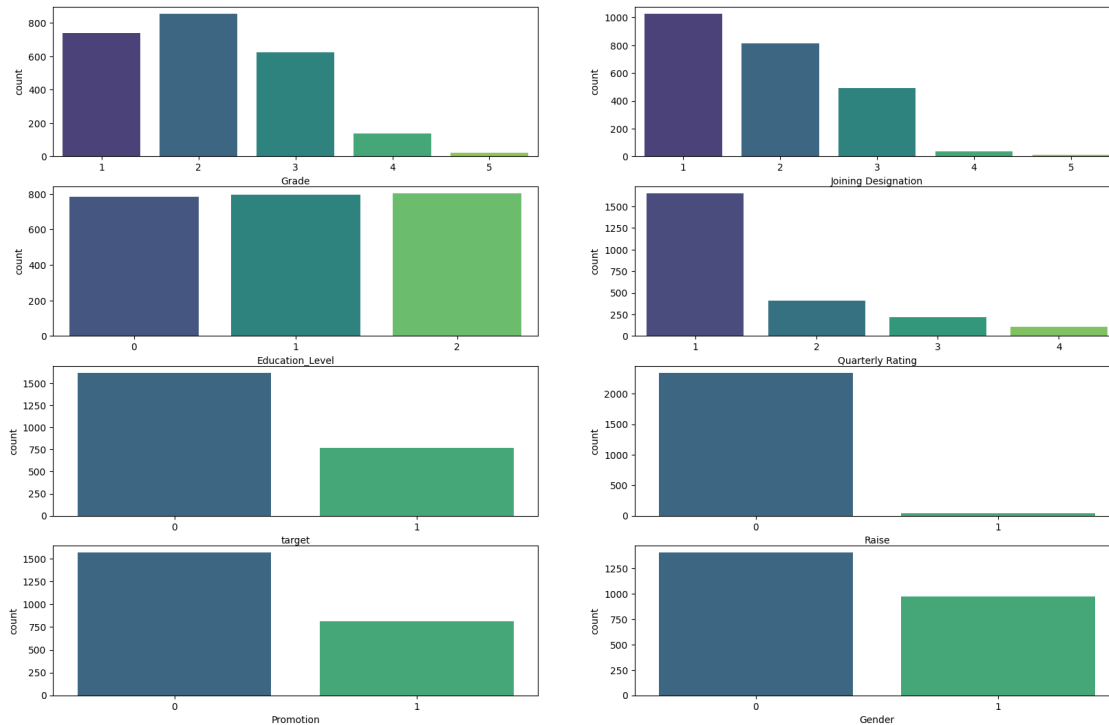
plt.subplot(4,2,5)
sns.countplot(x=ola1.target,palette='viridis')

plt.subplot(4,2,6)
sns.countplot(x=ola1.Raise,palette='viridis')

plt.subplot(4,2,7)
sns.countplot(x=ola1.Promotion,palette='viridis')
```

```
plt.subplot(4,2,8)
sns.countplot(x=ola1.Gender,palette='viridis')

plt.show()
```



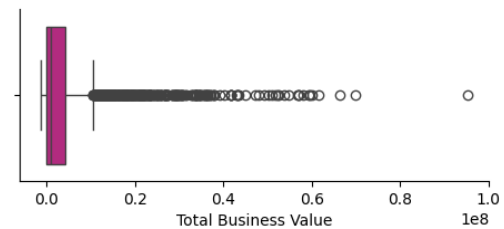
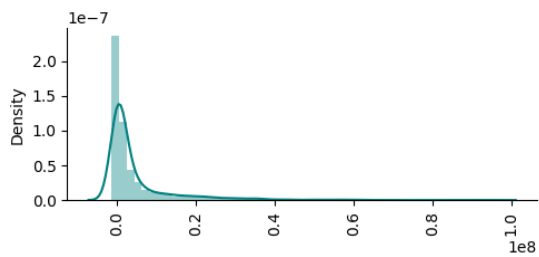
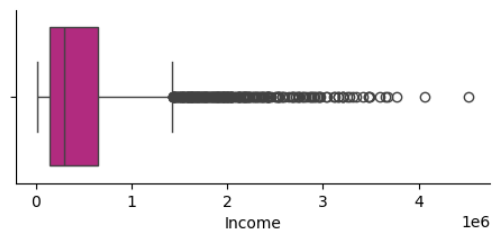
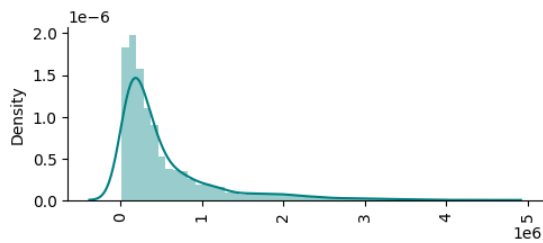
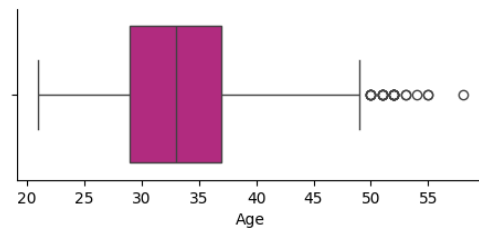
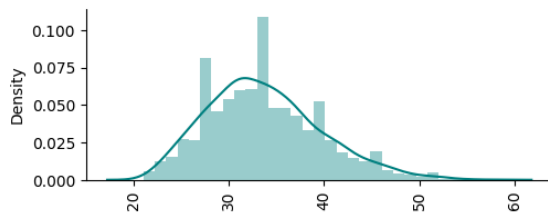
Observations:

- Between 21 years(min age) to 58(max age) years of age, maximum number of drivers are 32 years o meanwhile the age group between 31-41 years of age receives the maximum number of drivers.
- 58.9% of the Drivers are male.
- City C20 has been used by the most of the drivers.
- There are 3 Education levels and all of them alomst have the equal distribution of Drivers.
- Grade 2 has been received by most of the Drivers and then the count of grade keeps on falling.

```
[61]: a =ola1[['Age','Income','Total Business Value']]
for i in a:
    plt.figure(figsize=(12,2))
    plt.subplot(121)
    sns.distplot(x=ola1[i],color='teal')
    plt.title('')
```

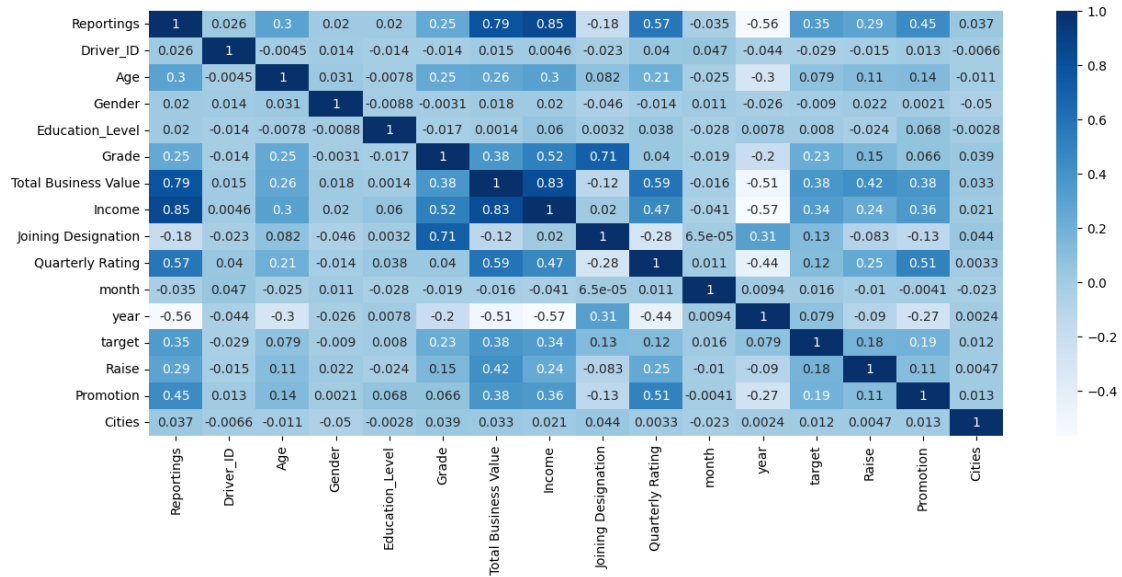
```
plt.xticks(rotation=90)

plt.subplot(122)
sns.boxplot(x=ola1[i],color='mediumvioletred')
plt.title('')
sns.despine()
plt.show()
```



0.3 Bivariate and multivariate

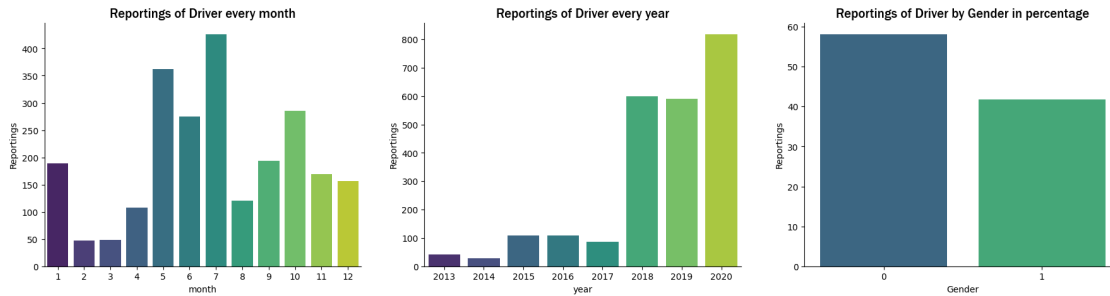
```
[71]: corr = ola1.corr()
plt.figure(figsize=(15,6))
sns.heatmap(corr,annot=True,cmap='Blues')
plt.show()
```



```
[72]: fig = plt.figure(figsize=(22,5))
ax = fig.add_subplot(1,3,1)
grouped_months = ola1.groupby(['month'])['Reportings'].count().reset_index()
sns.barplot(data=grouped_months,x='month',y='Reportings',palette='viridis')
plt.title('Reportings of Driver every month',fontname='Franklin Gothic Medium',
fontsize=15)

ax = fig.add_subplot(1,3,2)
grouped_years = ola1.groupby(['year'])['Reportings'].count().reset_index()
sns.barplot(x='year', y='Reportings', data=grouped_years,palette='viridis')
plt.title('Reportings of Driver every year',fontname='Franklin Gothic Medium',
fontsize=15)

ax = fig.add_subplot(1,3,3)
grouped_gender = ola1.groupby('Gender')['Reportings'].sum().reset_index()
grouped_gender['Reportings'] =(grouped_gender['Reportings']/sum(ola1.
Reportings)*100).round(2)
sns.barplot(x=grouped_gender['Gender'],y=
grouped_gender['Reportings'],palette='viridis')
plt.title('Reportings of Driver by Gender in percentage',fontname='Franklin
Gothic Medium', fontsize=15)
sns.despine()
sns.despine()
plt.show()
```

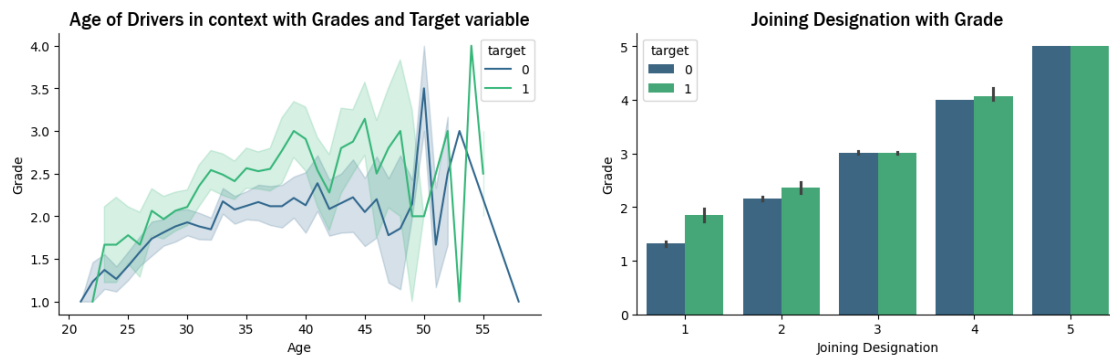



[73]: grouped_gender

```
[73]:   Gender  Reportings
      0      0      58.12
      1      1      41.88
```

```
[74]: fig = plt.figure(figsize=(15,4))
      ax = fig.add_subplot(1,2,1)
      sns.lineplot(x=ola1.Age,y=ola1.Grade,hue=ola1.target,palette='viridis')
      plt.title('Age of Drivers in context with Grades and Target_
      ↪variable',fontname='Franklin Gothic Medium', fontsize=15)

      ax = fig.add_subplot(1,2,2)
      sns.barplot(data=ola1, x="Joining Designation",
      ↪y="Grade",palette='viridis',hue='target')
      plt.title('Joining Designation with Grade',fontname='Franklin Gothic Medium',
      ↪fontsize=15)
      sns.despine()
      plt.show()
```

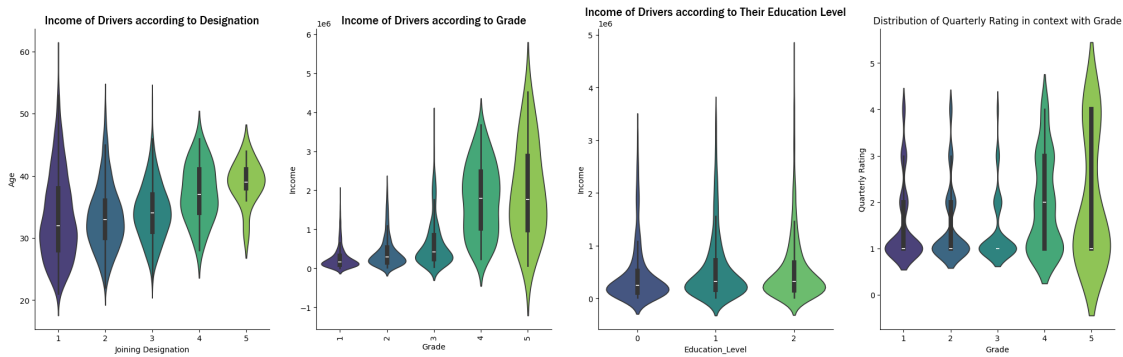


```
[75]: plt.figure(figsize=(25,7))
      plt.subplot(1,4,1)
```

```

sns.violinplot(y=ola1.Age,x=ola1['Joining Designation'],palette='viridis')
plt.title('Income of Drivers according to Designation',fontname='Franklin_
↳Gothic Medium', fontsize=15)
plt.subplot(1,4,2)
sns.violinplot(x=ola1.Grade,y=ola1.Income,palette='viridis')
plt.title('Income of Drivers according to Grade',fontname='Franklin Gothic_
↳Medium', fontsize=15)
plt.xticks(rotation=90)
plt.subplot(1,4,3)
sns.violinplot(x=ola1.Education_Level,y=ola1.Income,palette='viridis')
plt.title('Income of Drivers according to Their Education_
↳Level',fontname='Franklin Gothic Medium', fontsize=15)
plt.subplot(1,4,4)
sns.violinplot(x=ola1['Grade'],y=ola1["Quarterly Rating"],palette='viridis')
plt.title('Distribution of Quarterly Rating in context with Grade')
sns.despine()
sns.despine()
plt.show()

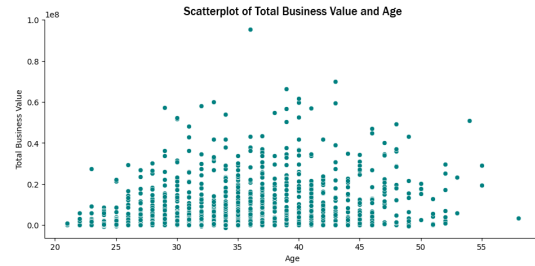
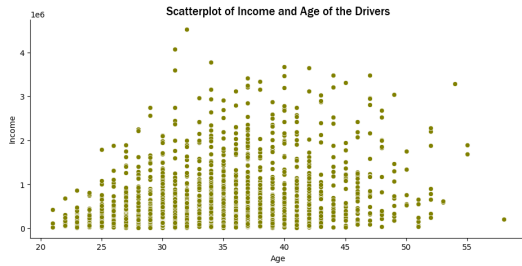
```



```

[76]: plt.figure(figsize=(25,5))
plt.subplot(1,2,1)
sns.scatterplot(x=ola1.Age,y=ola1.Income,color='olive')
plt.title('Scatterplot of Income and Age of the Drivers',fontname='Franklin_
↳Gothic Medium', fontsize=15)
plt.subplot(1,2,2)
sns.scatterplot(x=ola1.Age,y=ola1['Total Business Value'],color='teal')
plt.title('Scatterplot of Total Business Value and Age',fontname='Franklin_
↳Gothic Medium', fontsize=15)
sns.despine()
plt.show()

```



```
[77]: grouped_gender = ola1.groupby('Gender')['Income'].sum().reset_index()
grouped_education = ola1.groupby('Education_Level')['Income'].sum().
    ↪reset_index()
grouped_grade = ola1.groupby('Grade')['Income'].sum().reset_index()
grouped_desig = ola1.groupby('Joining Designation')['Income'].sum().
    ↪reset_index()
grouped_QR = ola1.groupby('Quarterly Rating')['Income'].sum().reset_index()
grouped_target = ola1.groupby('target')['Income'].sum().reset_index()
grouped_raise = ola1.groupby('Raise')['Income'].sum().reset_index()
grouped_promote = ola1.groupby('Promotion')['Income'].sum().reset_index()
```

```
[80]: def donut_pie(values, labels, title):
    wedges, texts, autotexts = plt.pie(
        values,
        labels=labels,
        autopct='%1.1f%%',
        startangle=90,
        colors=sns.color_palette('flare'),
        labeldistance=1.1,      # Push labels outside
        pctdistance=0.75       # Keep % neatly inside
    )

    # Add donut hole
    hole = plt.Circle((0, 0), 0.5, facecolor='white')
    plt.gca().add_artist(hole)

    plt.title(title, fontsize=12)

plt.figure(figsize=(15, 8))

plt.subplot(3,3,1)
donut_pie(
    grouped_gender['Income'],
    grouped_gender['Gender'],
    'Income with respect to Gender'
)
```

```

plt.subplot(3,3,2)
donut_pie(
    grouped_education['Income'],
    grouped_education['Education_Level'],
    'Income with respect to Education Level'
)

plt.subplot(3,3,3)
donut_pie(
    grouped_grade['Income'],
    grouped_grade['Grade'],
    'Income with respect to Grade'
)

plt.subplot(3,3,4)
donut_pie(
    grouped_desig['Income'],
    grouped_desig['Joining Designation'],
    'Income with respect to Joining Designation'
)

plt.subplot(3,3,5)
donut_pie(
    grouped_QR['Income'],
    grouped_QR['Quarterly Rating'],
    'Income with respect to Quarterly Rating'
)

plt.subplot(3,3,6)
donut_pie(
    grouped_target['Income'],
    grouped_target['target'],
    'Income with respect to Target Variable'
)

plt.subplot(3,3,7)
donut_pie(
    grouped_raise['Income'],
    grouped_raise['Raise'],
    'Income with respect to Raise Given'
)

plt.subplot(3,3,8)
donut_pie(
    grouped_promote['Income'],
    grouped_promote['Promotion'],

```

```

    'Income with respect to Promotion Given'
)

sns.despine()
plt.tight_layout()
plt.show()

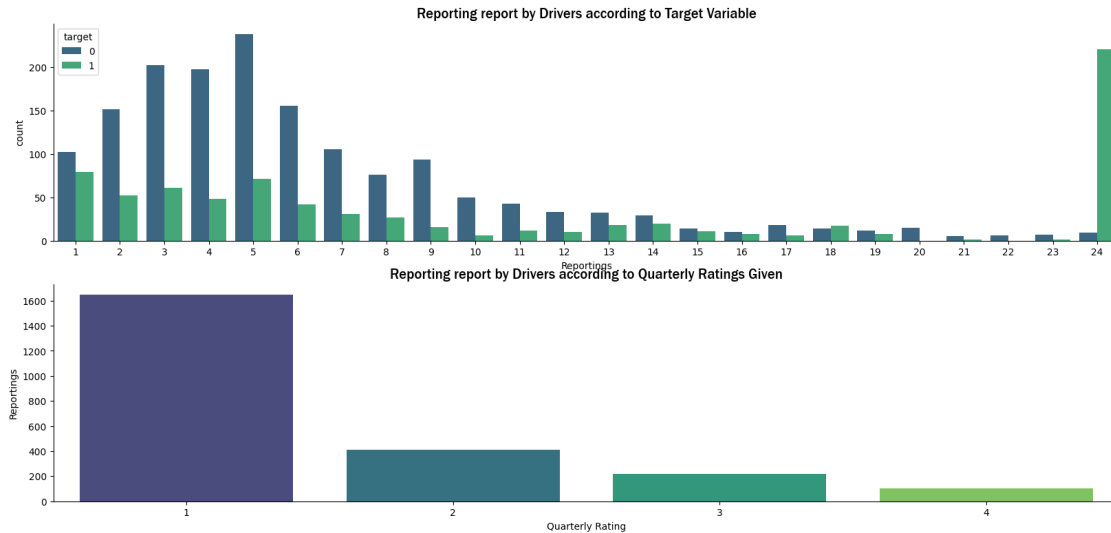
```



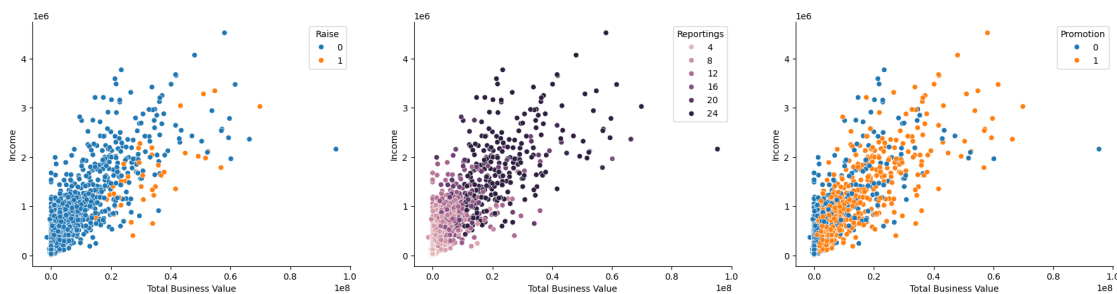
```

[81]: plt.figure(figsize=(20,9))
plt.subplot(2,1,1)
sns.countplot(x=ola1['Reportings'],hue=ola1.target,palette='viridis')
plt.title('Reporting report by Drivers according to Target_
↳Variable',fontname='Franklin Gothic Medium', fontsize=15)
plt.subplot(2,1,2)
grouped_rating = ola1.groupby('Quarterly Rating')['Reportings'].count().
↳reset_index()
sns.barplot(data = grouped_rating,y='Reportings',x='Quarterly_
↳Rating',palette='viridis')
plt.title('Reporting report by Drivers according to Quarterly Ratings_
↳Given',fontname='Franklin Gothic Medium', fontsize=15)
sns.despine()
plt.show()

```



```
[82]: plt.figure(figsize=(22,5))
plt.subplot(1,3,1)
sns.scatterplot(x=ola1['Total Business Value'],y=ola1.Income,hue=ola1.Raise)
plt.subplot(1,3,2)
sns.scatterplot(x=ola1['Total Business Value'],y=ola1.Income,hue=ola1.
↳Reportings)
plt.subplot(1,3,3)
sns.scatterplot(x=ola1['Total Business Value'],y=ola1.Income,hue=ola1.Promotion)
sns.despine()
plt.show()
```

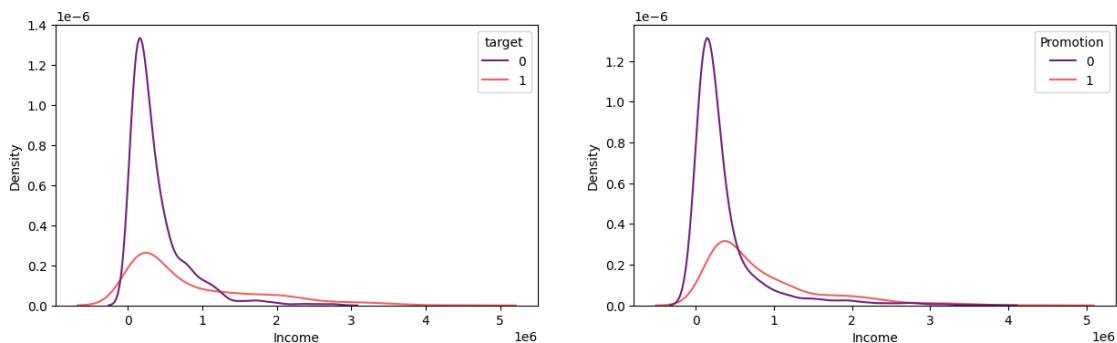


Observation:

- So we see that there are 57% male employees and 43% female employees.
- The percentages of employees with different education levels are almost same for level 1 & 2.
- 97.3% of the employees who did not get a raise.

- Almost 43% of the employees joined at lowest designation (1). 34% joined at level 2, 20% at level 3 and below 2% joined at higher levels.
- Majority (35%) of the employees currently are at designation level 2, followed by designation level 1 (31%) and 3 (26%). Less than 5% of the employees are currently in higher designations.
- Only 54.6% of the employees received a promotion, while 45.4% did not. However, only 2.6% received a raise in income.
- Number of employees has been increase with increase in year as well as number of reportings.
- The majority of the employees seem to be associated with city C20.
- Scatter plot of Income shows that Income increases with increase in age but after 45-50, we see a subtle decline.
- Scatter plot of Total Business Value shows an increase with increase in Age yet we notice a decline after 45.
- Income decreases with increase in Destination as about 4% of the employees hold higher designations.
- The median of the Income for employees having higher Grades is greater.
- Distribution of Income for employes at different Education level is about a change of 3-5% with level 0.
- Joining Designation Increases with increase in Grade.
- Max reporting days is 24 days.
- About 55% of the reportings of the employees has got Quarlerly Rating 1.
- Number of reportings increases with increase in Income as well as Total Business Value.

```
[83]: plt.figure(figsize=(15,4))
plt.subplot(1,2,1)
sns.kdeplot(x=ola1.Income,hue=ola1['target'],palette='magma')
plt.subplot(1,2,2)
sns.kdeplot(x=ola1.Income,hue=ola1['Promotion'],palette='magma')
plt.show()
```



OUTLIER TREATMENT

```
[84]: ola1.describe().T
```

```
[84]:
```

	count	mean	std	min	25%	\
Reportings	2381.0	8.023520e+00	6.783590e+00	1.0	3.0	
Driver_ID	2381.0	1.397559e+03	8.061616e+02	1.0	695.0	
Age	2381.0	3.366317e+01	5.983375e+00	21.0	29.0	
Gender	2381.0	4.103318e-01	4.919972e-01	0.0	0.0	
Education_Level	2381.0	1.007560e+00	8.162900e-01	0.0	0.0	
Grade	2381.0	2.096598e+00	9.415218e-01	1.0	1.0	
Total Business Value	2381.0	4.586742e+06	9.127115e+06	-1385530.0	0.0	
Income	2381.0	5.267603e+05	6.231633e+05	10883.0	139895.0	
Joining Designation	2381.0	1.820244e+00	8.414334e-01	1.0	1.0	
Quarterly Rating	2381.0	1.486350e+00	8.343483e-01	1.0	1.0	
month	2381.0	6.975220e+00	3.007801e+00	1.0	5.0	
year	2381.0	2.018536e+03	1.609597e+00	2013.0	2018.0	
target	2381.0	3.212936e-01	4.670713e-01	0.0	0.0	
Raise	2381.0	1.805964e-02	1.331951e-01	0.0	0.0	
Promotion	2381.0	3.427131e-01	4.747162e-01	0.0	0.0	
Cities	2381.0	1.533557e+01	8.371843e+00	1.0	8.0	

	50%	75%	max
Reportings	5.0	10.0	24.0
Driver_ID	1400.0	2100.0	2788.0
Age	33.0	37.0	58.0
Gender	0.0	1.0	1.0
Education_Level	1.0	2.0	2.0
Grade	2.0	3.0	5.0
Total Business Value	817680.0	4173650.0	95331060.0
Income	292980.0	651456.0	4522032.0
Joining Designation	2.0	2.0	5.0
Quarterly Rating	1.0	2.0	4.0
month	7.0	10.0	12.0
year	2019.0	2020.0	2020.0
target	0.0	1.0	1.0
Raise	0.0	0.0	1.0
Promotion	0.0	1.0	1.0
Cities	15.0	22.0	29.0

```
[85]: len(ola1[ola1['Total Business Value'] < 1])
```

```
[85]: 729
```

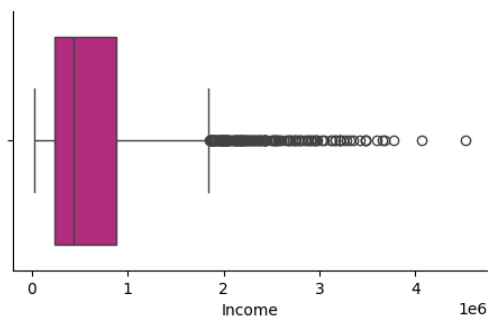
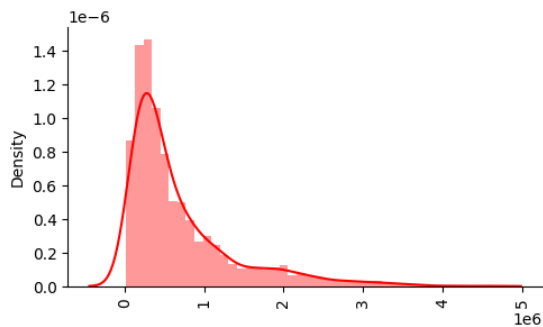
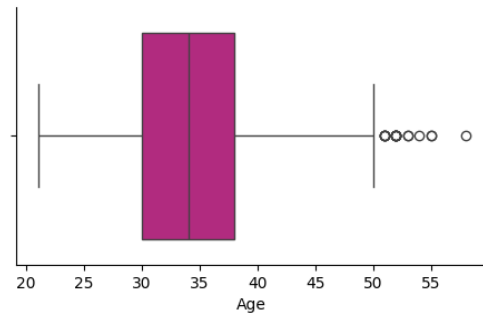
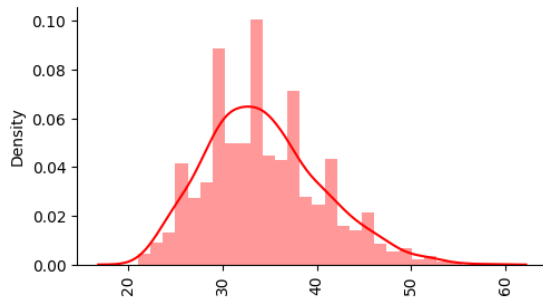
Observation:

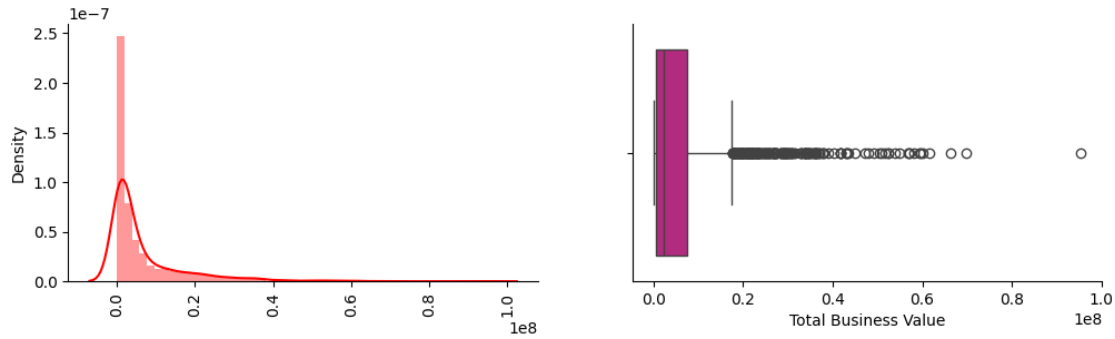
- As we can notice Total Business Value column has some values in negative.
- We consider them as outlier which will affect the results of the our machine learning model.

- Considering the parts of datasets that has Total Business Value > 1.
- There are exactly 729 Driver having Total Business Value that less than 1.

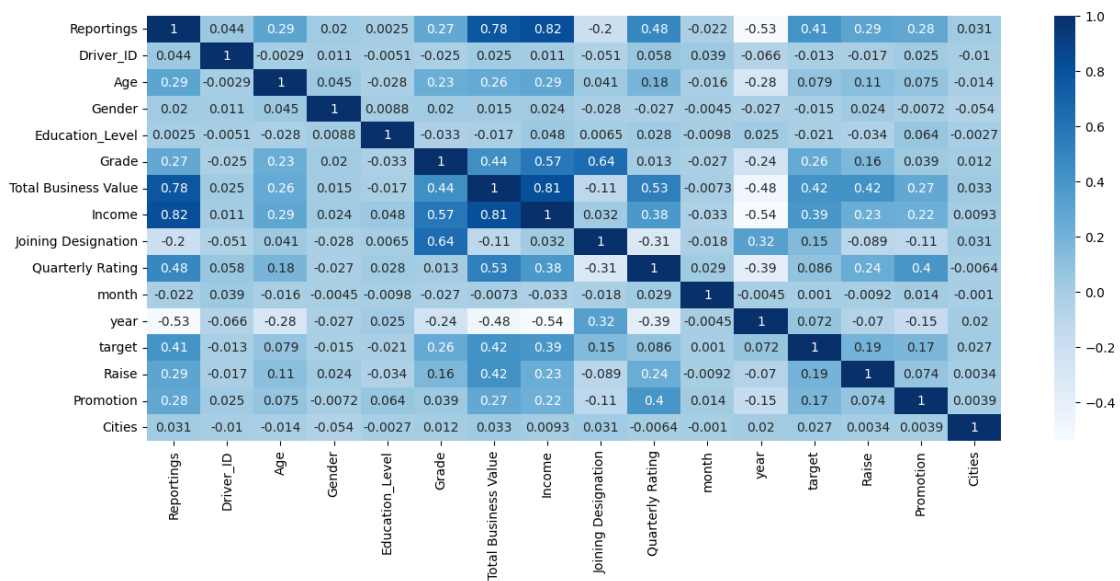
```
[86]: ola1= ola1[ola1['Total Business Value'] > 1]
```

```
[87]: a =ola1[['Age','Income','Total Business Value']]
for i in a:
    plt.figure(figsize=(12,3))
    plt.subplot(121)
    sns.distplot(x=ola1[i],color='red')
    plt.xticks(rotation=90)
#     plt.figure(figsize=(9,5))
    plt.subplot(122)
    sns.boxplot(x=ola1[i],color='mediumvioletred')
    sns.despine()
    plt.show()
```





```
[89]: corr = ola1.corr()
plt.figure(figsize=(15,6))
sns.heatmap(corr,annot=True,cmap='Blues')
plt.show()
```



```
[90]: ola1.describe().T
```

```
[90]:
```

	count	mean	std	min	25%	\
Reportings	1652.0	1.026998e+01	6.967589e+00	1.0	5.0	
Driver_ID	1652.0	1.390315e+03	8.082919e+02	1.0	679.5	
Age	1652.0	3.432385e+01	6.190776e+00	21.0	30.0	
Gender	1652.0	4.158596e-01	4.930188e-01	0.0	0.0	
Education_Level	1652.0	1.030872e+00	8.093284e-01	0.0	0.0	
Grade	1652.0	2.144068e+00	9.719606e-01	1.0	1.0	
Total Business Value	1652.0	6.613094e+06	1.032794e+07	19580.0	663022.5	

Income	1652.0	6.864932e+05	6.814522e+05	20886.0	236652.5
Joining Designation	1652.0	1.759685e+00	8.395129e-01	1.0	1.0
Quarterly Rating	1652.0	1.700363e+00	9.237035e-01	1.0	1.0
month	1652.0	6.914044e+00	3.021205e+00	1.0	5.0
year	1652.0	2.018208e+03	1.730439e+00	2013.0	2018.0
target	1652.0	3.619855e-01	4.807202e-01	0.0	0.0
Raise	1652.0	2.602906e-02	1.592699e-01	0.0	0.0
Promotion	1652.0	4.933414e-01	5.001070e-01	0.0	0.0
Cities	1652.0	1.545278e+01	8.374318e+00	1.0	8.0

	50%	75%	max
Reportings	8.0	14.0	24.0
Driver_ID	1385.0	2097.0	2788.0
Age	34.0	38.0	58.0
Gender	0.0	1.0	1.0
Education_Level	1.0	2.0	2.0
Grade	2.0	3.0	5.0
Total Business Value	2242080.0	7418392.5	95331060.0
Income	428960.0	877151.0	4522032.0
Joining Designation	2.0	2.0	5.0
Quarterly Rating	1.0	2.0	4.0
month	7.0	9.0	12.0
year	2018.0	2020.0	2020.0
target	0.0	1.0	1.0
Raise	0.0	0.0	1.0
Promotion	0.0	1.0	1.0
Cities	16.0	23.0	29.0

0.4 ENSEMBLE LEARNING

Data Prepration: The Trade-Off In general while choosing a model, we might choose to look at precision and recall scores and choose while keeping the follwing trade-off on mind : -

- If we prioritize precision, we are going to reduce our false positives. This may be useful if our targeted retention strategies prove to be expensive. We don't want to spend unnecessarily on somebody who is not even going to leave in the first place. Also, it might lead to uncomfortable situation for the employee themselves if they are put in a situation where it is assumed that they are going to be let go/ going to leave.
- If we prioritize recall, we are going to reduce our false negatives. This is useful since usually the cost of hiring a new person is higher than retaining n experienced person. So, by reducing false negatives, we would be able to better identify those who are actually going to leave and try to retain them by appropriate measures (competitve remuneration, engagement program, etc).

```
[94]: !pip install xgboost
```

```
Requirement already satisfied: xgboost in
c:\users\dell\appdata\local\programs\python\python313\lib\site-packages (3.1.1)
```

Requirement already satisfied: numpy in
c:\users\dell\appdata\local\programs\python\python313\lib\site-packages (from
xgboost) (2.2.4)

Requirement already satisfied: scipy in
c:\users\dell\appdata\local\programs\python\python313\lib\site-packages (from
xgboost) (1.15.2)

[notice] A new release of pip is available: 24.3.1 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

```
[95]: from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV
```

```
[96]: X = ola1.drop('target',axis=1)
y = ola1['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
↳random_state= 42)
```

```
[97]: from sklearn.model_selection import learning_curve
def plot_learning_curve(estimator, X, Y, title):
    train_sizes, train_scores, test_scores, _, _ =
↳learning_curve(estimator,X,Y,return_times=True)
    fig, axes = plt.subplots(1, 1, figsize = (15, 5))
    axes.set_title(title)
    axes.plot
    axes.set_xlabel("Training examples")
    axes.set_ylabel("Score")
    train_scores_mean = np.mean(train_scores, axis=1)
    train_scores_std = np.std(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)
    test_scores_std = np.std(test_scores, axis=1)
    # Plot learning curve
    # 32
    axes.grid()
    axes.fill_between(
```

```

train_sizes,
train_scores_mean - train_scores_std,
train_scores_mean + train_scores_std,
alpha=0.1,
color="r",
)
axes.fill_between(
train_sizes,
test_scores_mean - test_scores_std,
test_scores_mean + test_scores_std,
alpha=0.1,
color="g",
)
axes.plot(
train_sizes, train_scores_mean, "o-", color="r", label="Training score"
)
axes.plot(
train_sizes, test_scores_mean, "o-", color="g", label="Cross-validation_
↪score"
)
axes.legend(loc="best")
plt.show()

```

[98]: X.head()

```

[98]:
  Reportings  Driver_ID  Age  Gender  Education_Level  Grade  \
0           3          1   28       0                2       1
2           5          4   43       0                2       2
3           3          5   29       0                0       1
4           5          6   31       1                1       3
7           6         12   35       0                2       1

    Total Business Value  Income  Joining Designation  Quarterly Rating  month  \
0           1715580    172161                1                2       12
2           350000    328015                2                1       11
3           120360    139104                1                1       12
4           1265000    393640                3                1       12
7           2607180    168696                1                4        1

    year  Raise  Promotion  Cities
0  2018      0           0      23
2  2019      0           0      13
3  2019      0           0       9
4  2020      0           1      11
7  2019      0           1      23

```

```
[99]: ss= StandardScaler()
      ss.fit_transform(X_train)
```

```
[99]: array([[ -0.61446611, -1.09640018,  1.70794584, ..., -0.16737851,
          1.023749   , -0.04979913],
        [ 1.93718866, -1.32951199,  1.54780698, ..., -0.16737851,
        -0.97680193, -0.5247786  ],
        [-0.18919032, -1.0914666  ,  0.26669606, ..., -0.16737851,
          1.023749   ,  1.25639439],
        ...,
        [-0.75622471,  0.03585718, -1.49483144, ..., -0.16737851,
        -0.97680193, -0.88101319],
        [ 0.51960268,  1.32105562, -1.33469258, ..., -0.16737851,
          1.023749   , -1.59348238],
        [-0.33094892,  0.60815284, -0.69413712, ..., -0.16737851,
        -0.97680193, -0.28728886]], shape=(1321, 15))
```

```
[100]: from sklearn.model_selection import cross_validate
```

```
[101]: valid1 = cross_val_score(LogisticRegression(),X,y,cv=5)
      print('Logistic Regression:',valid1.round(2))
      print('Mean:',valid1.mean())
      valid2 = cross_val_score( DecisionTreeClassifier(),X,y,cv=5)
      print('Decision Tree:',valid2.round(3))
      print('Mean:',valid2.mean())
      valid3 = cross_val_score(RandomForestClassifier(),X,y,cv=5)
      print('RandomForestClassifier():',valid3.round(2))
      print('Mean:',valid3.mean())
      valid4 = cross_val_score(GradientBoostingClassifier(),X,y,cv=5)
      print('GradientBoostingClassifier:',valid4.round(3))
      print('Mean:',valid4.mean())
      valid5 =cross_val_score(XGBClassifier(),X,y,cv=5)
      print('XGBoostClassifier:',valid1.round(2))
      print('Mean:',valid5.mean())
```

Logistic Regression: [0.73 0.76 0.75 0.75 0.78]

Mean: 0.7548548933443193

Decision Tree: [0.843 0.855 0.864 0.848 0.818]

Mean: 0.8456376453355305

RandomForestClassifier(): [0.89 0.9 0.89 0.87 0.9]

Mean: 0.8898251396136592

GradientBoostingClassifier: [0.888 0.915 0.882 0.852 0.848]

Mean: 0.8770887118923373

XGBoostClassifier: [0.73 0.76 0.75 0.75 0.78]

Mean: 0.873461503250023

0.5 MACHINE LEARNING MODEL

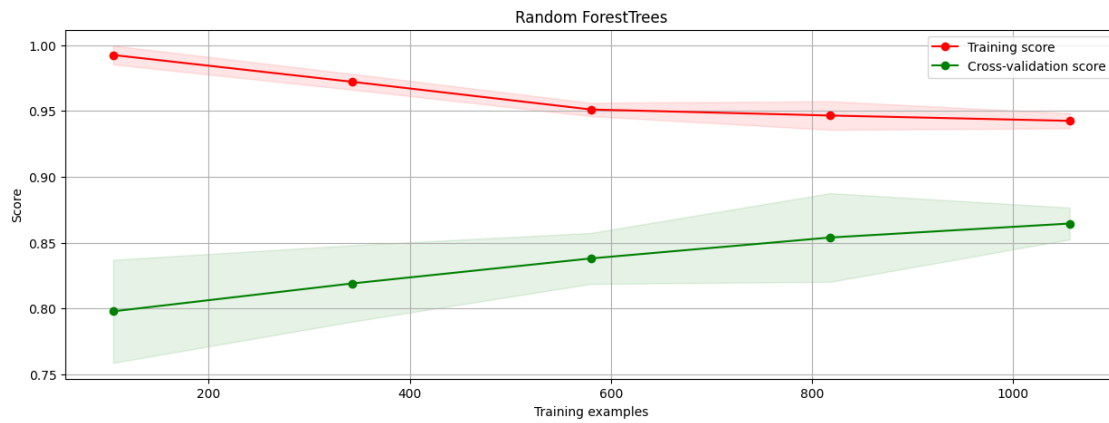
WITHOUT THE TREATMENT OF CLASS IMBALANCE

Random Forest Classifier

```
[106]: rf_clf1 = RandomForestClassifier(criterion='gini',max_depth=7,max_features='sqrt',n_estimators=10)
rf_clf1.fit(X_train,y_train)

[106]: RandomForestClassifier(max_depth=7, n_estimators=10)

[107]: plot_learning_curve(rf_clf1, X_train, y_train, "Random ForestTrees")
```



```
[108]: y_pred = rf_clf1.predict(X_test)
proba = rf_clf1.predict_proba(X_test)[: ,1]
print("Train data accuracy:",rf_clf1.score(X_train, y_train))
print("Test data accuracy:",rf_clf1.score(X_test,y_test))
print('Accuracy of the model:', accuracy_score(y_test, y_pred))
print("ROC-AUC score test dataset: ", roc_auc_score(y_test, proba))
print('-'*70)
print(classification_report(y_test, y_pred))
print('-'*70)
cm1 = (confusion_matrix(y_test, y_pred))
print('Confusion Metrix')
print(confusion_matrix(y_test, y_pred))
```

```
Train data accuracy: 0.9348978046934141
Test data accuracy: 0.8580060422960725
Accuracy of the model: 0.8580060422960725
ROC-AUC score test dataset: 0.9378993299049401
```

```
-----
precision    recall  f1-score   support
```

0	0.87	0.91	0.89	207
1	0.84	0.77	0.80	124

accuracy			0.86	331
macro avg	0.85	0.84	0.85	331
weighted avg	0.86	0.86	0.86	331

Confusion Metrix

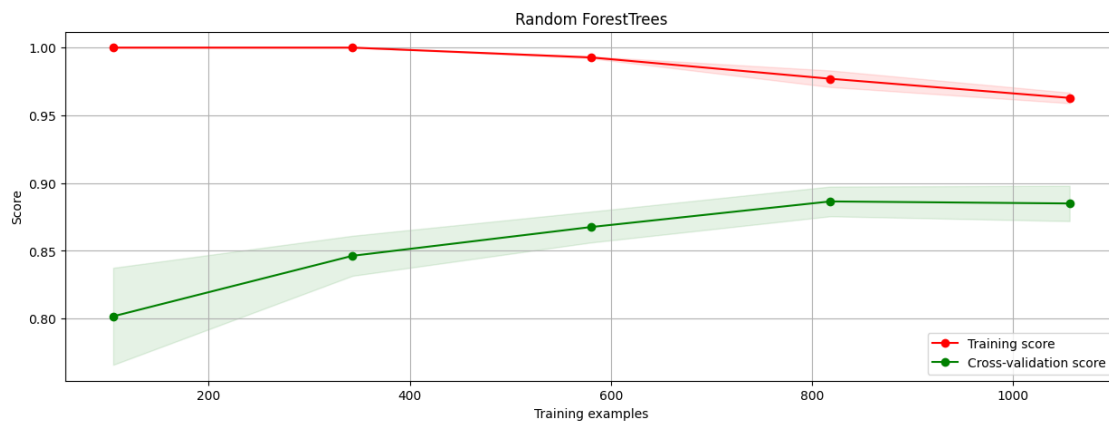
```
[[189  18]
 [ 29  95]]
```

```
[109]: rf_clf_imp1 = rf_clf1.feature_importances_
```

XG Boosting Classifier

```
[110]: gbc1 = GradientBoostingClassifier()
gbc1.fit(X_train, y_train)
y_pred = gbc1.predict(X_test)
proba = gbc1.predict_proba(X_test)[: , 1]
```

```
[112]: plot_learning_curve(gbc1, X_train, y_train, "Random ForestTrees")
```



```
[113]: gbc_clf_imp1 = gbc1.feature_importances_
```

```
[114]: print('Train Score : ', gbc1.score(X_train, y_train))
print('Test Score : ', gbc1.score(X_test, y_test))
print('Accuracy Score : ', accuracy_score(y_test, y_pred))
print("ROC-AUC score test dataset: ", roc_auc_score(y_test, proba))
print('-'*60)
print(classification_report(y_test, y_pred))
print('-'*60)
print('Confusion Matrix')
```



```
cm2 = (confusion_matrix(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print('-'*60)
```

Train Score : 0.9553368660105981
 Test Score : 0.9003021148036254
 Accuracy Score : 0.9003021148036254
 ROC-AUC score test dataset: 0.9491195262583763

```
-----
              precision    recall  f1-score   support

     0       0.91         0.94         0.92         207
     1       0.89         0.84         0.86         124

 accuracy                   0.90         331
 macro avg       0.90         0.89         0.89         331
weighted avg       0.90         0.90         0.90         331

-----
```

```
-----
Confusion Matrix
[[194  13]
 [ 20 104]]
-----
```

CLASS IMBALANCE TREATMENT

```
[115]: plt.figure(figsize=(15,4))
sns.countplot(x=y_train,palette='Set2')
plt.title('Class Imbalance in the Data')
plt.show()
```



```
[116]: (y_train.value_counts()*100)/len(y_train)
```

```
[116]: target
0      64.118092
```

```
1    35.881908
Name: count, dtype: float64
```

```
[117]: from imblearn.over_sampling import SMOTE
```

```
[118]: smot = SMOTE(random_state=42)
X_train_smot, y_train_smot = smot.fit_resample(X_train, y_train.ravel())
```

```
[119]: X_train_smot.shape, y_train_smot.shape
```

```
[119]: ((1694, 15), (1694,))
```

```
[120]: X_test.shape, y_test.shape
```

```
[120]: ((331, 15), (331,))
```

```
[121]: from collections import Counter
c = Counter(y_train_smot)
print(c)
```

```
Counter({np.int64(0): 847, np.int64(1): 847})
```

Random Forest Classifier

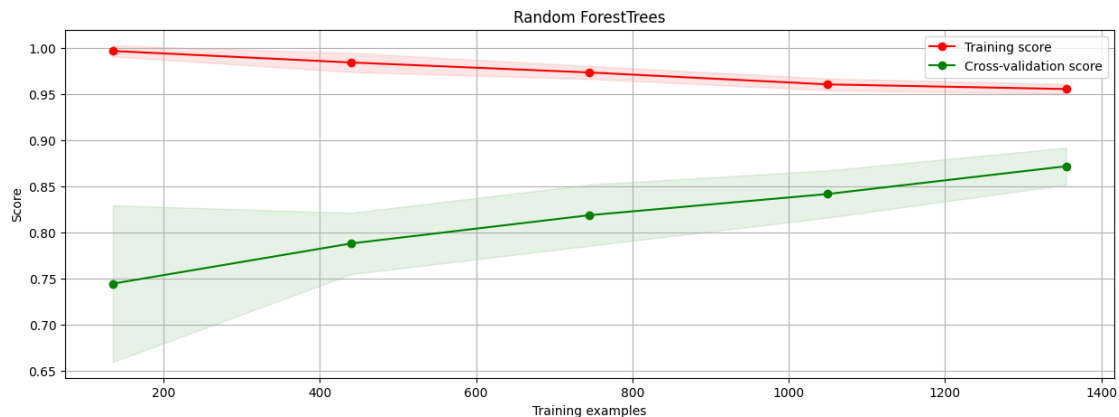
```
[122]: clf = RandomForestClassifier()
clf.fit(X_train_smot, y_train_smot)
```

```
[122]: RandomForestClassifier()
```

```
[123]: clf = RandomForestClassifier(criterion='gini', max_depth=8,
                                   max_features='sqrt', n_estimators=19)
clf.fit(X_train_smot, y_train_smot)
```

```
[123]: RandomForestClassifier(max_depth=8, n_estimators=19)
```

```
[124]: plot_learning_curve(clf, X_train_smot, y_train_smot, "Random ForestTrees")
```



```
[126]: y_pred = clf.predict(X_test)
print('-'*70)
print(classification_report(y_test, y_pred))
print('-'*70)
print('Confusion Metrix')
cm3 = confusion_matrix(y_test, y_pred)
print(confusion_matrix(y_test, y_pred))
```

```
-----
              precision    recall  f1-score   support

     0       0.92      0.86      0.89        207
     1       0.79      0.87      0.83        124

 accuracy                   0.86        331
 macro avg       0.85      0.87      0.86        331
 weighted avg    0.87      0.86      0.87        331

-----
```

```
Confusion Metrix
[[178  29]
 [ 16 108]]
```

```
[127]: rf_clf_imp2= clf.feature_importances_
```

Gradient Boosting

```
[128]: gbc2 = GradientBoostingClassifier()
gbc2.fit(X_train_smot, y_train_smot)
y_pred1 = gbc2.predict(X_test)
gbc_clf_imp2 = gbc2.feature_importances_
print('-'*60)
print(classification_report(y_test, y_pred1))
print('-'*60)
cm4 = confusion_matrix(y_test, y_pred1)
print('Confusion Metrix')
print(cm4)
print('-'*60)
```

```
-----
              precision    recall  f1-score   support

     0       0.93      0.89      0.91        207
     1       0.83      0.90      0.86        124

 accuracy                   0.89        331

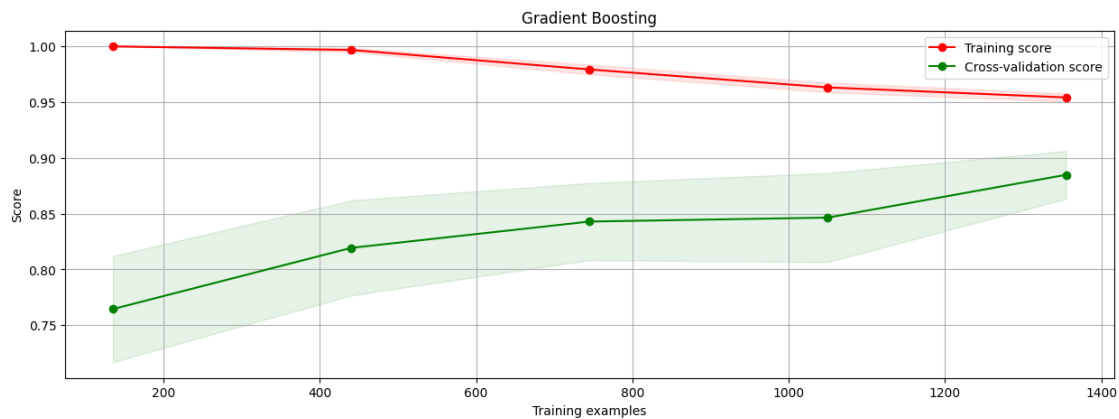
-----
```

macro avg	0.88	0.89	0.89	331
weighted avg	0.90	0.89	0.89	331

Confusion Matrix

```
[[185  22]
 [ 13 111]]
```

```
[129]: plot_learning_curve(gbc2, X_train_smot, y_train_smot, "Gradient Boosting")
```



```
[130]: data1 = pd.DataFrame({'Column_Name':X.columns,
                             'RandomForestClassifier':rf_clf_imp1,
                             'XGBClassifier':gbc_clf_imp1})
```

```
[131]: data2 = pd.DataFrame({'Column_Name':X.columns,
                             'RandomForestClassifier':rf_clf_imp2,
                             'XGBClassifier':gbc_clf_imp2})
```

```
[132]: data1
```

```
[132]:
```

	Column_Name	RandomForestClassifier	XGBClassifier
0	Reportings	0.310323	0.421251
1	Driver_ID	0.027980	0.012013
2	Age	0.036500	0.007798
3	Gender	0.001583	0.001911
4	Education_Level	0.004546	0.000578
5	Grade	0.012231	0.000488
6	Total Business Value	0.092723	0.124586
7	Income	0.115264	0.017669
8	Joining Designation	0.046993	0.006300
9	Quarterly Rating	0.056120	0.028193
10	month	0.016427	0.005197

11	year	0.211906	0.344124
12	Raise	0.019052	0.000000
13	Promotion	0.025337	0.018855
14	Cities	0.023016	0.011039

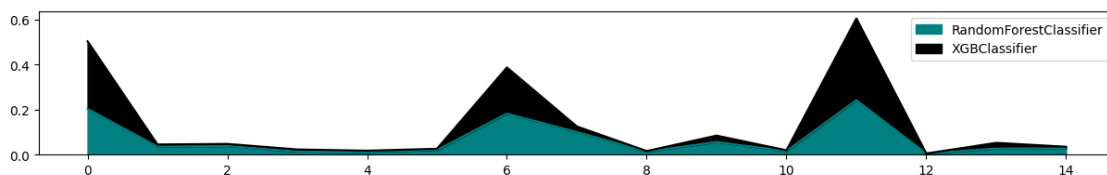
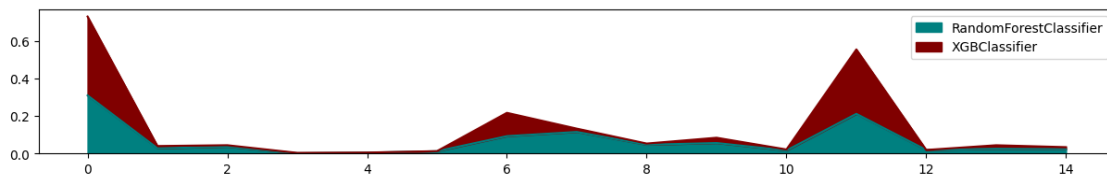
[133]: data2

	Column_Name	RandomForestClassifier	XGBClassifier
0	Reportings	0.203980	0.300373
1	Driver_ID	0.035864	0.009665
2	Age	0.038527	0.009567
3	Gender	0.014216	0.009140
4	Education_Level	0.013037	0.004790
5	Grade	0.021065	0.005402
6	Total Business Value	0.182706	0.205282
7	Income	0.100998	0.024849
8	Joining Designation	0.013791	0.002461
9	Quarterly Rating	0.056419	0.028597
10	month	0.016277	0.003503
11	year	0.242630	0.363173
12	Raise	0.005447	0.000000
13	Promotion	0.027370	0.025453
14	Cities	0.027675	0.007745

```
[134]: data1.plot(kind="area", figsize = (15,2),color=['teal','maroon'])

data2.plot(kind="area", figsize = (15,2),color=['teal','black'])

plt.show()
```



```
[135]: tp1,fp1,fn1,tn1 =cm1[0][0],cm1[0][1],cm1[1][0],cm1[1][1]
        tp2,fp2,fn2,tn2 =cm2[0][0],cm2[0][1],cm2[1][0],cm2[1][1]
        tp3,fp3,fn3,tn3 =cm3[0][0],cm3[0][1],cm3[1][0],cm3[1][1]
        tp4,fp4,fn4,tn4 =cm4[0][0],cm4[0][1],cm4[1][0],cm4[1][1]
        precision1 = tp1/(tp1+fp1)
        recall1 = tp1/(tp1+fn1)
        precision2 = tp2/(tp2+fp2)
        recall2 = tp2/(tp2+fn2)
        precision3 = tp3/(tp3+fp3)
        recall3 = tp3/(tp3+fn3)
        precision4 = tp4/(tp4+fp4)
        recall4 = tp4/(tp4+fn4)
        f1_1 = (2*precision1*recall1)/(precision1+recall1)
        f1_2 = (2*precision2*recall2)/(precision2+recall2)
        f1_3 = (2*precision3*recall3)/(precision3+recall3)
        f1_4 =(2*precision4*recall4)/(precision4+recall4)
```

```
[136]: df = pd.DataFrame({'Model':
    ↪ ['RandomForest','GradientBoosting','RandomForest','GradientBoosting'],
        'Class': ['imbalanced','imbalanced','balanced','balanced'],
        'True_pos': [tp1,tp2,tp3,tp4],
        'Fal_pos': [fp1,fp2,fp3,fp4],
        'Fal_neg': [fn1,fn2,fn3,fn4],
        'True_neg': [tn1,tn2,tn3,tn4],
        'F1_score%': [f1_1*100,f1_2*100,f1_3*100,f1_4*100],
        'Precision%':
    ↪ [precision1*100,precision2*100,precision3*100,precision4*100],
        'Recall%': [recall1*100,recall2*100,recall3*100,recall4*100]})
```

```
[137]: df
```

```
[137]:
```

	Model	Class	True_pos	Fal_pos	Fal_neg	True_neg	\
0	RandomForest	imbalanced	189	18	29	95	
1	GradientBoosting	imbalanced	194	13	20	104	
2	RandomForest	balanced	178	29	16	108	
3	GradientBoosting	balanced	185	22	13	111	

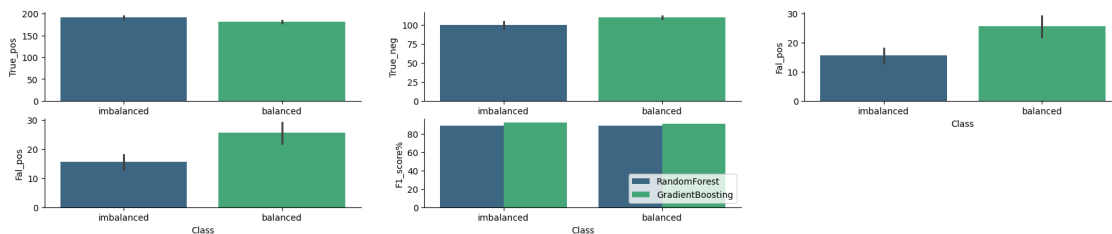
	F1_score%	Precision%	Recall%
0	88.941176	91.304348	86.697248
1	92.161520	93.719807	90.654206
2	88.778055	85.990338	91.752577
3	91.358025	89.371981	93.434343

```
[138]: plt.figure(figsize=(22,4))
        plt.subplot(2,3,1)
        sns.barplot(x=df.Class,y=df.True_pos,palette='viridis')
        # plt.show()
```

```

plt.subplot(2,3,2)
sns.barplot(x=df.Class,y=df.True_neg,palette='viridis')
# plt.show()
plt.subplot(2,3,3)
sns.barplot(x=df.Class,y=df.Fal_pos,palette='viridis')
# plt.show()
plt.subplot(2,3,4)
sns.barplot(x=df.Class,y=df.Fal_pos,palette='viridis')
plt.subplot(2,3,5)
sns.barplot(x=df.Class,y=df['F1_score%'],palette='viridis',hue=df.Model)
plt.legend(loc='lower right')
sns.despine()
plt.show()

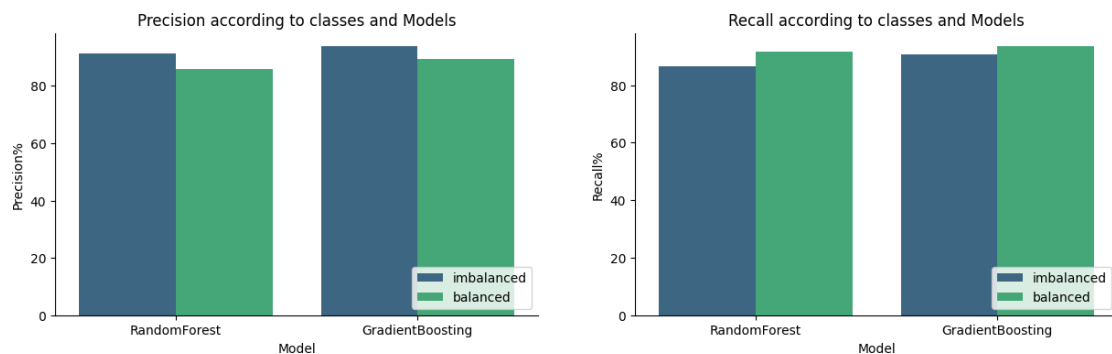
```



```

[139]: plt.figure(figsize=(15,4))
plt.subplot(1,2,1)
sns.barplot(x=df.Model,y=df['Precision%'],hue=df.Class,palette='viridis')
plt.title('Precision according to classes and Models')
plt.legend(loc='lower right')
plt.subplot(1,2,2)
sns.barplot(x=df.Model,y=df['Recall%'],hue=df.Class,palette='viridis')
plt.title('Recall according to classes and Models')
plt.legend(loc='lower right')
sns.despine()
plt.show()

```



0.6 Insights:

1. Workforce Demographics & Behavior

- Majority of Ola drivers are male (58%), while females form around 42%.
- Drivers typically fall in the age group of 31–41, which is also the most active and productive segment.
- Income and business value both rise with age, but peak around 45–50 and decline afterwards, likely due to physical work fatigue.

2. Education, Grade & Designation

- Education Levels 1 and 2 have almost equal representation, meaning education isn't a strong differentiator in hiring.
- Most drivers join at Designation 1 (low level) → 43%, and fewer join at higher positions.
- Current grade distribution shows Grade 2 has the largest chunk (~35%), followed by Grade 1 and 3.

3. Promotions, Raises & Performance

- Only ~2.6% drivers received a salary raise, which is extremely low.
- ~55% received at least one promotion.
- Quarterly Ratings show most drivers get a rating of 1, which might indicate:
- Strict rating criteria
- Performance issues
- Or rating inflation/deflation problems

4. Business Value & Income

- Total Business Value shows a heavy positive skew with extreme outliers.
- Removing negative values (>700 drivers) improved correlations & model performance.
- Higher grades, better ratings, and more experience correspond to higher income.

5. Seasonal & Temporal Trends

- July has the highest driver onboarding.
- Very low onboarding during February and March.
- Significant jump in hiring post-2017 → rapid business expansion.

0.7 Recommendations:

1. Strengthen Driver Retention Through Performance-Linked Interventions

- Implement targeted retention programs for drivers showing low business value, low reporting frequency, or declining ratings.

- Introduce structured performance coaching for at-risk drivers highlighted by the churn model.
 - Create an early-warning dashboard based on predictive model outputs.
2. Revise Compensation Structure to Improve Retention
 - Establish a minimum annual increment policy, as only 2–3% of drivers currently receive a raise.
 - Introduce milestone-based incentives tied to business value, customer satisfaction, and reporting consistency.
 - Offer higher incentives during peak months to match demand.
 3. Redesign Promotion Framework to Increase Motivation
 - Simplify the promotion pathway and communicate clear, objective criteria tied to performance metrics.
 - Ensure promotion decisions are data-backed and occur at regular intervals.
 - Offer small intermediate-level designations to prevent stagnation for long-tenure drivers.
 4. Improve Support Programs for Older Drivers
 - Provide age-friendly work options, such as reduced shift lengths, flexible timings, or lighter routes.
 - Launch annual health screening and wellness initiatives for drivers over 45.
 - Implement productivity maintenance programs to counter declining business value in higher age brackets.
 5. Reevaluate Quarterly Rating Mechanism
 - Conduct an audit of the rating system, as >50% drivers consistently receive Rating 1.
 - Retrain city managers/supervisors to standardize rating distribution and eliminate rating bias.
 - Align ratings with measurable KPIs to ensure fairness and consistency.
 6. Optimize Hiring Strategy Based on Seasonal Trends
 - Increase recruitment and training capacity in June–August, the period with highest driver onboarding.
 - Deploy additional hiring campaigns during low-join months (Feb–Mar) to maintain operational balance.
 - Align regional supply planning with city-wise demand trends.
 7. Strengthen Driver Engagement in High-Activity Cities
 - Prioritize engagement and reward programs in top driver cities (e.g., C20), which significantly contribute to total business value.
 - Invest in localized support teams to reduce attrition in high-density clusters.
 8. Enhance Data Collection & Monitoring Infrastructure

- Integrate customer feedback, cancelled rides, and route history into the driver performance dataset.
- Use telematics data to track driving behavior and personalize training programs.
- Implement unified dashboards for real-time attrition risk monitoring.

9. Implement Model-Driven Retention Actions

- Deploy the Gradient Boosting churn model (highest F1-score ~91%) in production for monthly churn forecasting.
- Prioritize high-recall strategies to reduce false negatives—ensuring drivers likely to leave are never missed.
- Conduct quarterly retraining of the model with updated data.

10. Strengthen the Driver Relationship Lifecycle

- Offer onboarding incentives for new drivers during early months when churn risk is high.
- Launch loyalty rewards for long-tenure drivers completing 2, 3, and 5-year milestones.
- Provide periodic skill-development workshops to increase driver confidence and retention.

[]: